Ref. Ares(2023)6548588 - 27/09/2023

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D7.7 Software tools on public Git repository (first release)

Work Package: WP7

Authors: Arabinda Ghosh, Sadegh Soudjani

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1st October 2022

Duration: 36 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D7.7
Issue: 3.0
Issue Date: 26.09.2023
Prepared by: MPI-SWS
Approved: Yes
Dissemination Status: Public
History Chart:

Issue	Date (DD/MM/YYYY)	Updated Sections	Cause of Change	Implemented by
1.0	30.08.2023	All sections	New Document	Arabinda Ghosh
2.0	10.09.2023	All sections	Revised the text and modified the details	Sadegh Soudjani
3.0	26.09.2023	Section 2.3	Added content	Arabinda Ghosh

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

List of Figures **3**

1 Introduction **4**
1.1 Deliverable Description 4
1.2 Relation to Other SymAware Deliverables 4

2 First Release of the Software Tool on Public Git Repository **5**
2.1 Achieved Objectives 5
2.2 Structure of the Repository 5
2.3 Description of the Tool 5
2.4 Code Administration and Further Developments 8

3 Conclusion **12**

List of Figures

2.1 Screenshot of the Gitlab repository *EICSymAware* containing the first version of the software. 6
2.2 Screenshot of the license file. 6
2.3 Screenshot of the readme file. 7
2.4 Screenshot depicting a segment of the code. 7
2.5 A screenshot of *Agent* class. 9
2.6 A screenshot of *dynamics* function. 10
2.7 A screenshot of *SituationAwareness* class. 10
2.8 A screenshot of *Perception* class. 11
2.9 A screenshot of *Knowledge* class. 11
2.10 A screenshot of *CommunicationSocket* class. 11

1. Introduction

1.1 Deliverable Description

This deliverable marks a significant milestone in the SymAware project, as we are releasing our first software tool on a public Git repository. The tool represents the culmination of dedicated research, development, and collaboration aimed at addressing the project's objectives and contributes to the broader scientific and technological community. The objectives of this deliverable related to the software release are the following:

- The release of the software tool in public domain demonstrates transparency, showcasing the project's progress and adherence to EU guidelines for open access and open-source dissemination of project outcomes.
- The release will invite other researchers, developers, and stakeholders to collaborate on the project.
- Hosting the software on a public Git repository can help the members of the project build a community, which is also a key objective of Work package (WP) 7.
- Publicly releasing the software encourages peer review and feedback from experts in the field.
- The software tool will allow others to reproduce and verify your project's results, which is important for scientific and research projects. It enables other researchers to build upon the existing work and contribute to the advancement of knowledge.

1.2 Relation to Other SymAware Deliverables

The work package on dissemination and exploitation (WP7) aims to establish a methodology that leverages the networks of stakeholders and fosters cooperation with other work packages to gather information and achieve broad dissemination of the project outputs. Additionally, this strategy includes efforts to evaluate various options for ensuring the portal's long-term sustainability and usability beyond the project's end date. WP7 consists of four tasks, which are

- Plan for dissemination and exploitation including communication activities;
- Data management plan;
- Public website;
- Software tools on public Git repository.

2. First Release of the Software Tool on Public Git Repository

2.1 Achieved Objectives

A public Gitlab repository named *EICSymAware* has been created and the software tools are available in the same repository. The repository can be accessed through the following link

https://gitlab.mpi-sws.org/sadegh/eicsymaware/

The primary objective of the repository is to disseminate the results stemming from the research, development, and collaborative efforts carried out within this project. The culmination of this task aims to gather insights and contributions from the various work packages that explore diverse aspects of awareness, as well as from WP5, which focuses on use cases. The repository will unify the execution of all computational methods into a unified toolkit, employing the progressively enhanced logical framework and the Multi-Agent System (MAS) model established in earlier phases.

2.2 Structure of the Repository

The repository currently can be accessed by anyone with the link. All the project members have the access to add and modify the content of the repository. A screenshot of the *EICSymAware* repository is shown in Fig. 2.1. The repository currently has three files:

- **License file**: This contains a BSD 3-Clause License as promised in the deliverable D7.4 Data Management Plan (refer to Fig. 2.2).
- **Readme file**: This summarizes the objective of the project, code of conduct for making changes in the existing code for the project members, and information of the contact person (refer to Fig. 2.3).
- **Code**: The first version of the python code titled 'rw_code.py' (refer to Fig. 2.4).

2.3 Description of the Tool

In this section, a brief description of the developed tool is presented with a few screenshot of the code. Development of the tool is ongoing based on the continuous research and collaborations among the partners. The first version of the python code is available in the file titled 'rw_code.py'.

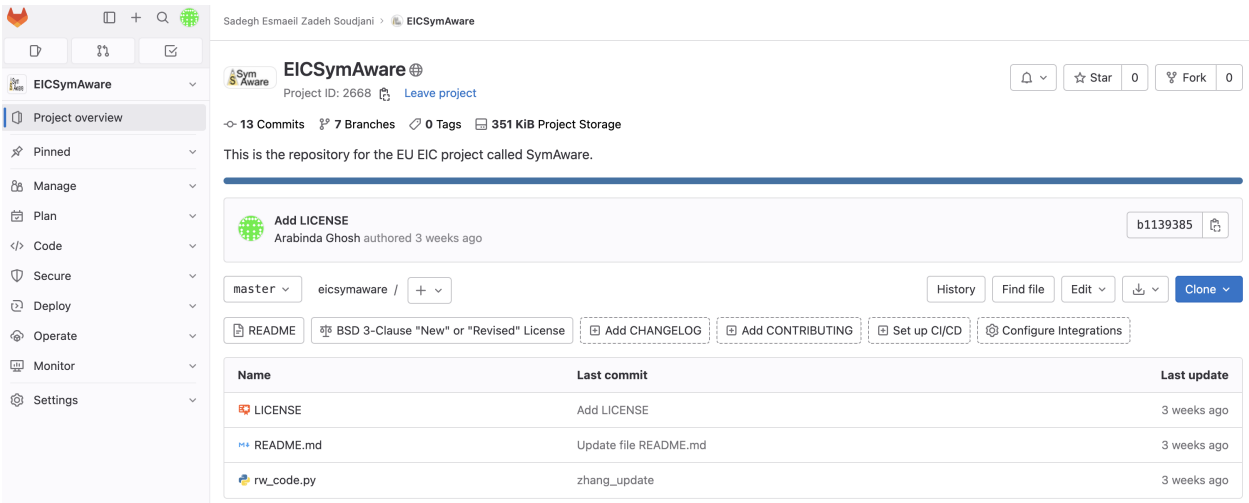

Figure 2.1: Screenshot of the Gitlab repository *EICSymAware* containing the first version of the software.

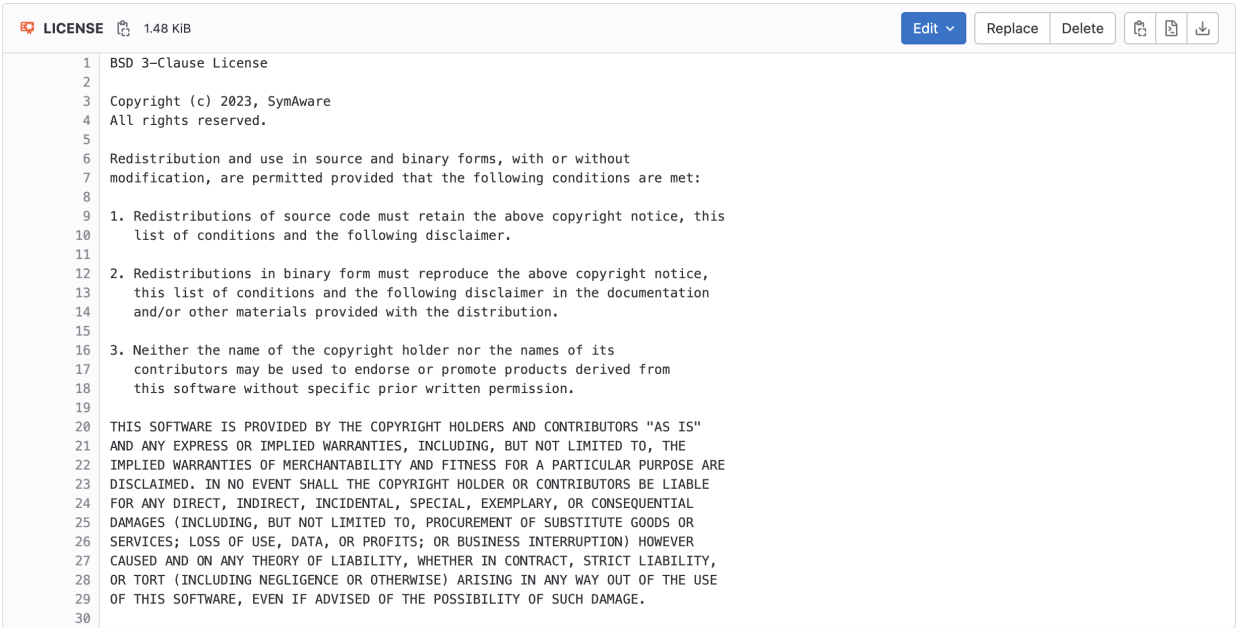

Figure 2.2: Screenshot of the license file.

On the onset, we have the *Agent* class which has attributes like IDs (own and neighbor's), state, situational awareness, communication information, control inputs. A snapshot of the *Agent* class is shown in Fig. 2.5. The system dynamics will be encoded through the fundtion *dynamics* as shown in Fig. 2.6. The *SituationAwareness* class is shown in Fig. 2.7. It has attributes like the state, intent, uncertainty and risk. The perception and estimation module is represented through *Perception* class as shown in Fig. 2.8. The knowledge module is represented through *Knowledge* class as shown

README.md

SymAware Project 🚗

Welcome to Symaware

SymAware addresses the fundamental need for a new conceptual framework for awareness in multi-agent systems (MASs) that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans.

The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions

Code of Conduct

Contributons to the project are allowed upon consent from the project managers. In order to make a contributiion, we highly recommend following the coding advised proposed from the Google Python Coding standards

Contact :phone:

- Gregorio Marchesini (gremar@kth.se)
- Arabinda Ghosh (arabinda@mpi-sws.org)
- Zengjie Zhang (z.zhang3@tue.nl)

Figure 2.3: Screenshot of the readme file.

```python


def dynamics(current_state, control_input):

    # single integrator
    next_state = control_input * STEPPING_TIME

    return next_state


class SituationAwareness:
    """
    The Class For Situational Awareness

    Attributes
    ----------
    sa_state            (int)   : the internal state of situation awareness
    sa_intent           (int)   : the intent variable of situation awareness
    uncertainty_measure (float) : the uncertainty measure
    risk_measure        (float) : the risk measure
    """

    uncertainty_measure = 0
    risk_measure = 0

    def __init__(self, initial_sa_state, initial_sa_intent):
        self.sa_state = initial_sa_state
        self.sa_intent = initial_sa_intent

    def update_uncertainty_measure(self):
        self.uncertainty_measure = 0

    def update_risk_measure(self):
        self.risk_measure = 0

```

Figure 2.4: Screenshot depicting a segment of the code.

in Fig. 2.9. Finally, the communication module is represented through *CommunicationSocket* class as shown in Fig. 2.10. It has attributes like agent's ID, state, knowledge, perception, control input, situational awareness.

2.4 Code Administration and Further Developments

MPI-SWS along with KTH and TUE have developed the first version of the code. All the partners will contribute significantly in developing the future versions of the software. The frequency of updates will depend on the project activities.

```python
class Agent:
    """Agent class

    Attributes
    ---------
    ID                  (int)           : the id of the agent
    neighbour_list      (list of int)   : the id list of neighbours
    agent_state         (numpy.array)   : the physical state of the agent
    sa                  (class SituationAwareness): the situation awareness object
    transmitted_socket  (class CommunicationSocket) : the transmitted communication socket of this agent
    received_sockets    (list of class CommunicationSocket) : the received communication sockets from the neighbours
    control_input       (numpy.array)   : the control input of the agent
    """

    ID = 0
    neighbour_list = []
    received_sockets = []

    def __init__(self, agent_id, neighbours, initial_agent_state, initial_sa_state, initial_sa_intent, initial_goal):
        self.ID = agent_id
        self.neighbour_list = neighbours
        self.agent_state = initial_agent_state
        # collect messages and update SA
        self.sa = SituationAwareness(initial_sa_state=initial_sa_state, initial_sa_intent=initial_sa_intent)
        self.control_input = self.planning_and_control(self.sa, initial_goal)
        self.transmitted_socket = CommunicationSocket(agent_id=self.ID,
                                                      agent_state=self.agent_state,
                                                      agent_control=self.control_input)

    def perform_simulate(self, agent_goal):
        """perfrom simulation of the agent dynamics over time"""

        self.estimation_and_perception()
        self.knowledge_awareness_update()

        self.sa.update_uncertainty_measure()
        self.sa.update_risk_measure()
        self.control_input = self.planning_and_control(self.sa, agent_goal)
        self.state_step(self.control_input)

    def state_step(self, control_input):

        self.agent_state = self.agent_state + dynamics(self.agent_state, control_input)

    def knowledge_awareness_update(self):
        """updates current knowledge broadcast message and returns updated knowledge"""

        # update current internal knowledge (TODO)

    def estimation_and_perception(self):
        """estimate/ perceive agent state and state of the neighbouring agents"""

        # perform estimation and perception (TODO)

    def planning_and_control(self, situation_awareness, control_goal):
        """computes plan and control input for the current state"""

        control_input = 0.3 * (control_goal - self.agent_state)  # simple P controller to make a showcase
        return control_input

    def transmitted_message(self):
        # update Situational Awareness for transmission
        self.transmitted_socket.agent_situation_awareness = self.sa
        self.transmitted_socket.socket_agent_control = self.control_input
        self.transmitted_socket.socket_agent_state = self.agent_state
        return self.transmitted_socket
```

Figure 2.5: A screenshot of *Agent* class.

```python
## System Dynamics

def dynamics(current_state, control_input):

    # single integrator
    next_state = control_input * STEPPING_TIME

    return next_state
```

Figure 2.6: A screenshot of *dynamics* function.

```python
## Situational Awareness

class SituationAwareness:
    """
    The Class For Situational Awareness

    Attributes
    ----------
    sa_state            (int)   : the internal state of situation awareness
    sa_intent           (int)   : the intent variable of situation awareness
    uncertainty_measure (float) : the uncertainty measure
    risk_measure        (float) : the risk measure
    """

    uncertainty_measure = 0
    risk_measure = 0

    def __init__(self, initial_sa_state, initial_sa_intent):
        self.sa_state = initial_sa_state
        self.sa_intent = initial_sa_intent

    def update_uncertainty_measure(self):
        self.uncertainty_measure = 0

    def update_risk_measure(self):
        self.risk_measure = 0

```

Figure 2.7: A screenshot of *SituationAwareness* class.

```python
## Perception module

class Perception:

    # This class is to be defined (TO DO)
    pass
```

Figure 2.8: A screenshot of *Perception* class.

```python
## Knowledge Module

class Knowledge:

    # This class is to be defined (TO DO)
    pass
```

Figure 2.9: A screenshot of *Knowledge* class.

```python
## Communication module

class CommunicationSocket:
    """
    The Class For Communication Sockets

    Attributes
    ----------
    agent_id                (int) : the id of the agent owning this socket
    agent_knowledge         (class Knowledge) : the knowledge transmitted by the agent
    agent_perception        (class Perception) : the perception made by the agent
    agent_situation_awareness(class SituationAwareness) : the situation awareness inferred by the agent
    socket_agent_control           (numpy.array) : the control input of this agent
    socket_agent_state             (numpy.array) : the state of this agent

    """
    socket_agent_id = 0
    agent_knowledge = Knowledge()
    agent_perception = Perception()
    agent_situation_awareness = SituationAwareness(initial_sa_state=0, initial_sa_intent=0)

    def __init__(self, agent_id, agent_state, agent_control):
        self.socket_agent_id = agent_id
        self.socket_agent_state = agent_state
        self.socket_agent_control = agent_control
```

Figure 2.10: A screenshot of *CommunicationSocket* class.

3. Conclusion

In conclusion, the completion and release of the first version of the software tool on the public Git repository represents a significant achievement in our project's development journey. Through dedicated effort, meticulous coding, and collaborative spirit, we have successfully reached this milestone, making our tools accessible to a wider audience and contributing to the broader community. In the rest of this project, we plan to discuss the tool development with other projects in the portfolio and establish collaboration or deploy similar good practices.